

Information and Network Security

Study Guide

Dr. Rahul Sharma
CSE, PIT
Parul University

6.1 Asymmetric Ciphers Prime Numbers.....	3
---	---

6.1 Asymmetric Ciphers: Prime Numbers

Asymmetric ciphers, also known as public-key cryptographic systems, rely heavily on the properties of prime numbers for secure communication. Unlike symmetric encryption, where the same key is used for both encryption and decryption, asymmetric encryption uses a pair of keys: a public key (used for encryption) and a private key (used for decryption). The security of such systems hinges on the computational difficulty of certain mathematical problems, primarily the factorization of large prime numbers.

Prime numbers — integers greater than 1 that have no positive divisors other than 1 and themselves — are foundational to asymmetric cryptography. Their importance stems from their use in algorithms like RSA (Rivest-Shamir-Adleman), which is one of the most widely used asymmetric encryption methods. In RSA, two large prime numbers are randomly selected and multiplied to produce a product called the modulus. This modulus is part of both the public and private keys. The difficulty of factoring this large product into its original prime components ensures the strength of the encryption.

The process begins by selecting two large primes, typically hundreds of digits long. These primes are used to calculate:

- The modulus $n=p \times q$
- Euler's totient function $\phi(n)=(p-1)(q-1)$

The security of RSA and similar systems lies in the fact that while it is easy to multiply large primes to get n , it is computationally infeasible to factor n back into its constituent primes, especially when the primes are sufficiently large (e.g., 2048 bits). This is known as the integer factorization problem, which lacks an efficient (polynomial-time) solution.

With advancements in quantum computing, traditional asymmetric systems may become vulnerable. Algorithms like Shor's algorithm could factor large numbers efficiently, challenging the security of RSA. This has led to ongoing research in post-quantum cryptography.

Prime numbers play a critical role in the construction and security of asymmetric ciphers. Their mathematical properties make them ideal for generating secure cryptographic keys and ensure that encrypted communication remains confidential and authentic.

Next Steps

- Study efficient algorithms for generating large prime numbers (e.g., Miller-Rabin primality test).
- Explore the RSA algorithm in detail through step-by-step implementation.
- Learn about other asymmetric algorithms like ECC and ElGamal.
- Analyze the security assumptions behind prime-based encryption (e.g., factorization difficulty).
- Research quantum computing's impact on asymmetric cryptography (e.g., Shor's algorithm).
- Practice encrypting and decrypting messages using public and private keys in programming exercises.

References:

Books:

- William Stallings, Cryptography and Network Security: Principles and Practice, 7th Edition, Pearson Education.
- Behrouz A. Forouzan, Cryptography and Network Security, McGraw-Hill Education.
- Douglas R. Stinson, Cryptography: Theory and Practice, CRC Press.
- NPTEL Online Courses, e.g., Cryptography and Network Security – by IIT Professors.
- RFC 3447, Public-Key Cryptography Standards (PKCS) #1: RSA Cryptography Specifications Version 2.1 – Internet Engineering Task Force (IETF)

6.2 Principles of Public-Key Cryptosystem

A public-key cryptosystem, also known as asymmetric cryptography, is a method of encryption that uses a pair of keys: a public key and a private key. The public key is shared openly and used for encryption, while the private key is kept secret and used for decryption. This key pair is mathematically related, but the private key cannot be feasibly derived from the public key, ensuring secure communication.

The fundamental principle behind a public-key cryptosystem is that it solves the problem of key distribution, which is a major challenge in symmetric key systems. In symmetric encryption, both the sender and receiver need to share the same secret key, which must be exchanged securely. In contrast, public-key systems eliminate this requirement because anyone can encrypt a message using the recipient's public key, but only the recipient can decrypt it using their private key.

Key components of a public-key cryptosystem include:

- **Key Generation:** A pair of keys (public and private) is generated using mathematical algorithms involving large prime numbers and modular arithmetic (e.g., in RSA).
- **Encryption:** The sender uses the receiver's public key to encrypt the message. Since the public key is accessible to anyone, this step does not compromise security.
- **Decryption:** The receiver uses their private key to decrypt the message. As only the receiver has access to the private key, the message remains confidential.
- **Authentication and Digital Signatures:** Public-key cryptosystems also support authentication. A sender can sign a message with their private key, and the receiver can verify the signature using the sender's public key. This proves the origin and integrity of the message.

Common algorithms used in public-key cryptography include RSA, Elliptic Curve Cryptography (ECC), and DSA (Digital Signature Algorithm). The security of these systems depends on the computational difficulty of certain mathematical problems, such as integer factorization or the discrete logarithm problem.

In conclusion, the principle of a public-key cryptosystem lies in the use of two mathematically related but distinct keys to ensure secure communication, confidentiality, and authentication

in digital interactions.

Next Steps

- Study the mathematical foundations behind key generation (e.g., modular arithmetic, prime numbers).
- Explore and implement public-key algorithms like RSA or ECC in programming languages such as Python or Java.
- Learn how digital signatures ensure message authenticity and integrity.
- Understand real-world applications, such as SSL/TLS, email encryption (PGP), and blockchain systems.

6.3 The RSA Algorithm

The RSA algorithm, named after its inventors Rivest, Shamir, and Adleman, is one of the most widely used public-key cryptographic systems. It is based on the mathematical difficulty of factoring large composite numbers and is used for both encryption and digital signatures.

Key Principles:

RSA is an asymmetric cryptographic algorithm, meaning it uses two different keys: a public key for encryption and a private key for decryption. The strength of RSA lies in the fact that, while it is easy to multiply two large prime numbers, it is extremely difficult to factor the resulting product back into its original primes a problem known as integer factorization.

RSA Key Generation Steps:

1. Choose two large prime numbers, p and q .
2. Compute $n = p \times q$; this value n is used as the modulus for both keys.
3. Calculate the totient: $\phi(n) = (p - 1)(q - 1)$.
4. Choose an integer e such that $1 < e < \phi(n)$ and $\gcd(e, \phi(n)) = 1$; e is the public exponent.
5. Compute the private exponent d , the modular inverse of e modulo $\phi(n)$:
$$d \equiv e^{-1} \pmod{\phi(n)}.$$

The public key is (e, n) and the private key is (d, n) .

Encryption and Decryption:

- **Encryption:** A sender uses the receiver's public key to encrypt a message M :
$$C = M^e \pmod{n}$$
- **Decryption:** The receiver uses their private key to decrypt the ciphertext C :
$$M = C^d \pmod{n}$$

Applications and Security:

RSA is widely used in secure data transmission, including SSL/TLS for internet security, email encryption, and digital signatures. Its security depends on the size of the key typically 2048 bits or more and the assumption that factoring large numbers is computationally infeasible with current technology.

In summary, the RSA algorithm provides a strong foundation for secure communication, relying on key mathematical principles and the practical impossibility of breaking the encryption without the private key.

Next Steps

- Practice RSA key generation and encryption/decryption with small numbers to understand the process.
- Implement RSA in a programming language like Python or Java for hands-on learning.
- Explore digital signatures using RSA to understand authentication and integrity.
- Study performance issues and security enhancements (e.g., padding schemes like OAEP).

6.4 Diffie Hellman Key Exchange

The Diffie-Hellman Key Exchange (DHKE) is a foundational cryptographic protocol that enables

two parties to securely establish a shared secret key over an insecure communication channel. This key can then be used for encrypted communication using symmetric encryption algorithms. Introduced in 1976 by Whitfield Diffie and Martin Hellman, this protocol marked the beginning of public-key cryptography.

The primary concept behind Diffie-Hellman is the difficulty of solving the discrete logarithm problem. It relies on modular arithmetic and a large prime number to create a secure exchange. Here's a step-by-step overview of how it works:

Public Parameters: Two global parameters are agreed upon by both users and can be shared publicly:

- A large prime number p
- A primitive root modulo p , called the generator g

Private Keys:

- User A selects a private key a , a random number less than p , and computes the public value $A = g^a \mod p$.
- User B selects a private key b , and computes the public value $B = g^b \mod p$.

Exchange and Shared Secret:

- User A sends A to User B, and User B sends B to User A.
- Now, each user computes the shared secret:
 - User A computes $s = B^a \mod p$
 - User B computes $s = A^b \mod p$
- Both users arrive at the same secret value s because $B^a \mod p = (g^b)^a \mod p = g^{ab} \mod p = g^{ba} \mod p = (g^a)^b \mod p = A^b \mod p$

Although the public keys A and B can be intercepted, the private keys a and b are never transmitted. The security of DHKE depends on the computational difficulty of deriving a or b from the public keys, a problem known as the discrete logarithm problem.

While DHKE provides secure key exchange, it does not authenticate the parties. Therefore, it is often combined with digital signatures or certificates in practical applications to prevent man-in-the-middle attacks.

Diffie-Hellman remains a core concept in cryptographic protocols and is widely used in secure communications, including HTTPS, VPNs, and messaging apps.

Next Steps

- The shared secret is converted into a symmetric key using a hash or key derivation function.
- This key is then used for encrypting and decrypting messages securely.
- Authentication (e.g., digital signatures) is added to prevent man-in-the-middle attacks.

6.5 Man in the Middle attack

A **Man-in-the-Middle (MITM) attack** is a type of cyberattack where an attacker secretly intercepts and possibly alters the communication between two parties who believe they are directly communicating with each other. It is a common and dangerous threat to data confidentiality and integrity, especially over unsecured or public networks.

In a MITM attack, the attacker places themselves between the sender and receiver. For example, if Alice wants to communicate securely with Bob, the attacker intercepts the messages and can relay them between Alice and Bob, making both believe they are talking directly to each other. Meanwhile, the attacker can view or even modify the transmitted data.

This can occur in several ways:

- **IP Spoofing:** The attacker tricks the target into thinking they're communicating with a trusted entity by falsifying IP addresses.
- **DNS Spoofing:** The attacker redirects the target to a fake website by corrupting the domain name system.
- **HTTPS Spoofing:** The attacker provides a fake SSL certificate to make a malicious site appear legitimate.
- **Wi-Fi Eavesdropping:** On public or unsecured Wi-Fi networks, attackers can intercept data traffic easily.

MITM in Cryptographic Protocols:

Protocols like the basic Diffie-Hellman Key Exchange are vulnerable to MITM attacks if not properly authenticated. In such a scenario, the attacker can establish separate keys with each party, decrypt and modify messages, then re-encrypt and forward them — all without detection.

Consequences:

- Theft of sensitive data (passwords, credit card info, etc.)
- Unauthorized access to systems
- Compromised communications and data integrity

Preventive Measures:

- Use of encryption (HTTPS, SSL/TLS)
- Authentication mechanisms like digital certificates and public key infrastructure (PKI)
- VPNs to encrypt network traffic
- Avoiding public Wi-Fi for sensitive transactions

Man-in-the-Middle attacks are serious threats that exploit the trust between two communicating parties. Implementing strong encryption, mutual authentication, and network security measures is crucial to prevent such attacks and ensure secure communication.

Next Steps

- **Immediately Terminate the Connection:** Disconnect the affected systems or users from the network to stop further data interception.
- **Identify the Attack Vector:** Analyze logs, traffic, or suspicious certificates to determine how the attacker gained access (e.g., spoofed DNS, fake Wi-Fi, compromised router).
- **Change All Compromised Credentials:** Reset passwords and update any authentication tokens or encryption keys that may have been exposed.
- **Update Security Protocols:** Implement HTTPS, SSL/TLS, VPNs, and enforce digital certificate validation to prevent future attacks.
- **Educate Users and Monitor Networks:** Train users to recognize suspicious activity (like browser certificate warnings) and monitor network traffic for anomalies.

